

Assignment 3: Bridge Pattern Implementation Report

Course: ShP-2216 Software Design Patterns | Trimester 4 | Instructor: Yerassyl Bekenov

Student Name:	Alibi Maksut	Group:	SE-2301	Topic:	Option A (Drawing)
Repository:	https://github.com/M1ska2021/Assignment3_Bridge				
Base Commit:	11ef76c1044c3b38667c3770c4925581593c8f98	Status:	All Checks Passed (7/7)		

1. Architectural Problem & Two Independent Dimensions

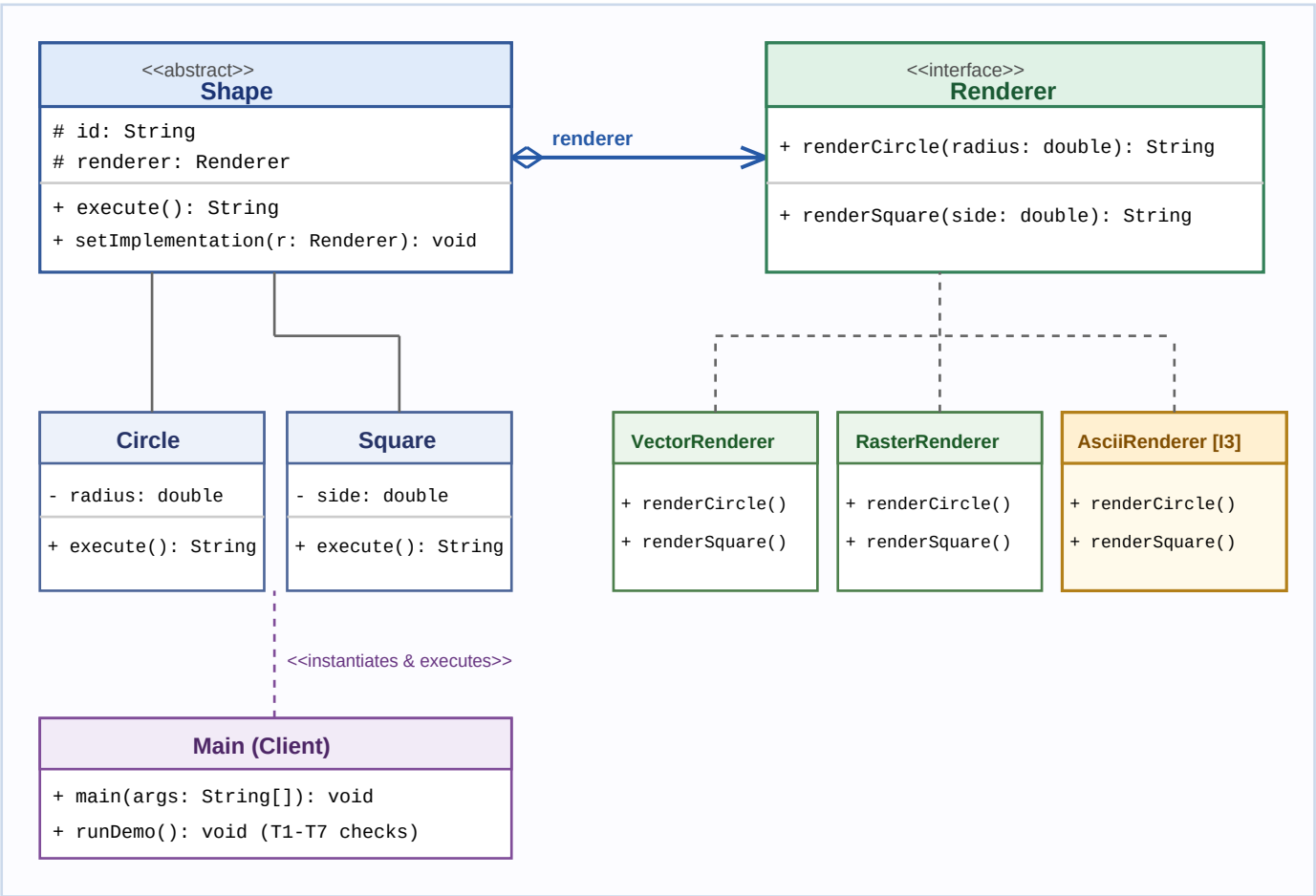
In object-oriented software design, class hierarchies often need to support multiple orthogonal dimensions of variation. In this assignment (Topic A: Drawing), our system incorporates two completely independent axes of change:

1. Domain Abstraction Dimension: Geometric shapes with domain-specific properties (Circle with radius, Square with side).
2. Low-level Implementor Dimension: Rendering platforms / formats (VectorRenderer, RasterRenderer, and AsciiRenderer).

The Subclass Explosion Problem (Cartesian Product): If inheritance were used to combine these two dimensions, supporting M shapes across N renderers would require M x N concrete subclasses (e.g., VectorCircle, RasterCircle, AsciiCircle, VectorSquare, RasterSquare, AsciiSquare). Introducing a 4th renderer would require modifying or adding classes across all shapes.

Bridge Pattern Solution: Bridge decouples the abstraction hierarchy from its implementation hierarchy by replacing direct inheritance with object composition. The abstract Shape class maintains an interface-typed reference to Renderer. This reduces the required number of classes from M x N to M + N (2 + 3 = 5), allowing shapes and renderers to vary, scale, and be tested independently.

2. UML Class Diagram



3. Five Clean Code Principles with Annotated Excerpts

Excerpt 1: Separation of Responsibilities (SRP)

```
public class Circle extends Shape {
    private final double radius;
    @Override
    public String execute() {
        return renderer.renderCircle(radius);
    }
}
```

Principle: Single Responsibility Principle (SRP).

Analysis: Circle only encapsulates geometric domain state (radius) and delegates visual representation to Renderer. Conversely, VectorRenderer only handles string formatting.

Benefit: High cohesion. Changes to rendering syntax or output formats never compromise core shape business logic.

Excerpt 2: Meaningful Names (Intention-Revealing)

```
boolean sameObject = (refBefore == refAfter);
boolean stateUnchanged = Objects.equals(
    idBefore, idAfter) &&
    (dataBefore == dataAfter);
boolean beforeMatch = "VECTOR circle radius=2"
    .equals(beforeResult);
```

Principle: Intention-Revealing & Meaningful Names.

Analysis: Variable names sameObject and stateUnchanged instantly communicate the verification contract of T5.

Method names renderCircle and setImplementation leave no ambiguity regarding their purpose.

Benefit: Minimizes cognitive overhead. Code is self-documenting.

Excerpt 3: Small Focused Methods (Do One Thing)

```
private String format(double value) {
    if (value == (long) value) {
        return String.valueOf((long) value);
    }
    return String.valueOf(value);
}
```

Principle: Small, Single-Purpose Methods.

Analysis: The helper format() method does exactly one thing: cleanly renders integers without trailing .0 decimals while preserving floating-point values when present.

Benefit: Trivial to unit test, debug, and reuse without contaminating high-level rendering orchestration.

Excerpt 4: Avoidance of Duplication (DRY)

```
public abstract class Shape {
    protected final String id;
    protected Renderer renderer;
    public void setImplementation(Renderer r) {
        this.renderer = Objects.requireNonNull(r);
    }
}
```

Principle: Don't Repeat Yourself (DRY).

Analysis: Bridge storage, constructor validation, and runtime implementation switching are consolidated in the Shape base. Subclasses Circle and Square inherit this mechanism cleanly.

Benefit: Eliminates duplicated bridge management code across present and future shape abstractions.

Excerpt 5: Encapsulation of State

```
protected final String id;
private final double radius;
public void setImplementation(Renderer renderer) {
    this.renderer = Objects.requireNonNull(renderer,
        "renderer must not be null");
}
```

Principle: Robust State Encapsulation.

Analysis: Identity (id) and geometric dimensions (radius, side) are declared immutable (final). The implementor reference is protected and mutated only through a validated method.

Benefit: Invariants are preserved throughout runtime switching, preventing null pointer exceptions and state corruption.

Summary of Clean Code Architecture

- No concrete implementor references exist anywhere in Shape, Circle, or Square.
- Abstractions interact with renderers strictly through the polymorphism of the Renderer interface.
- Zero type checks (instanceof) or casting operations are used, guaranteeing strict compliance with OOP principles.
- Every class is focused, readable, and under 35 lines of code, making maintenance effortless.

4. Independent Extension (I3) & Runtime Switching

Addition of AsciiRenderer (I3):

To satisfy the Open/Closed Principle (OCP), a third concrete implementor (AsciiRenderer) was introduced after the base commit (ea12080). AsciiRenderer implements the existing Renderer interface without requiring any modification to the abstraction hierarchy (Shape, Circle, Square) or the existing implementors (VectorRenderer, RasterRenderer).

Verification via Source Diff (extension.diff):

Running 'git diff BASE_COMMIT HEAD -- src/' proves that the only Java changes in src/ are the new AsciiRenderer.java class and updates to Main.java to run demonstration checks T6 and T7. Existing contracts and classes remained completely untouched.

Runtime Switching Demonstration (T5):

In test check T5, an existing Circle instance initialized with VectorRenderer is switched to RasterRenderer at runtime via circle.setImplementation(new RasterRenderer()). Reference equality (refBefore == refAfter) evaluates to true, verifying that the abstraction object identity and its internal domain data (id='C-05', radius=2.0) are completely preserved while the rendering strategy is replaced dynamically.

5. Design Trade-Off of Bridge & Comparison with Adapter

Architectural Trade-Off of the Bridge Pattern:

- Increased Upfront Structural Complexity: Introduces additional interfaces and indirection. In scenarios where only one implementation platform will ever exist, this introduces premature abstraction.
- Minor Indirection Overhead: Each high-level call requires dynamic dispatch through the bridge reference. In modern JVMs, monomorphic or bimorphic call sites are inlined by the JIT compiler, making runtime performance overhead negligible.
- Design Intent: Bridge is an UP-FRONT architectural pattern designed to let abstraction and implementation vary independently before code is finalized. Adapter is a RETROSPECTIVE pattern used to make incompatible existing interfaces work together.
- Structural Focus: Bridge connects two parallel class hierarchies via composition. Adapter wraps a single existing class/adaptee.

Key Differences: Bridge vs Adapter Pattern:

6. Demonstration Results (T1-T7 Verification Table)

Test	Setup / Target Action	Evidence / Result Observed	Status
T1	Circle with VectorRenderer	VECTOR circle radius=2	PASS
T2	Circle with RasterRenderer	RASTER circle radius=2	PASS
T3	Square with VectorRenderer	VECTOR square side=3	PASS
T4	Square with RasterRenderer	RASTER square side=3	PASS
T5	Runtime Switch (Vector -> Raster)	sameObject=true, stateUnchanged=true	PASS
T6	Circle with AsciiRenderer (I3)	ASCII circle radius=2	PASS
T7	Square with AsciiRenderer (I3)	ASCII square side=3	PASS

Overall Test Summary: 7 / 7 Demonstration Checks PASSED (100% Success Rate)

7. References

1. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
2. Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.
3. Bloch, J. (2018). Effective Java (3rd ed.). Addison-Wesley Professional.
4. Bekenov, Y. (2026). Lecture 4: Bridge Pattern. Astana IT University, Course ShP-2216 Software Design Patterns.